

**Universidad Nacional de Rosario Facultad de
Ciencias Exactas, Ingeniería y Agrimensura**

Tecnicatura Universitaria en Inteligencia Artificial

Trabajo práctico N°1

Procesamiento de Imágenes I

**Gimbatti Tomás, Paredes Julián, Nasini Rodrigo,
Longo Valentino**

05/10/2026

1. Introducción

El presente informe describe la resolución de los dos problemas planteados en el Trabajo Práctico N° 1. Para cada uno se presenta la descripción del ejercicio, el análisis de los problemas encontrados durante el desarrollo, las técnicas de procesamiento de imágenes utilizadas y las conclusiones obtenidas. Se incluyen capturas de los pasos intermedios generadas con el mismo código del proyecto.

2. Problema 1 – Ecualización local de histograma

2.1 Descripción del ejercicio

La ecualización de histograma redistribuye los niveles de intensidad para aprovechar todo el rango dinámico. En su versión global se calcula una única transformación con el histograma de toda la imagen. En la versión local se define una ventana de $M \times N$ que se desplaza píxel a píxel; en cada posición se calcula el histograma de los píxeles de la ventana, se obtiene la transformación de ecualización y se aplica sólo al píxel central.

Se pide: (a) implementar una función de ecualización local que reciba la imagen y el tamaño de ventana; (b) aplicarla a la imagen provista para descubrir los detalles ocultos en cada zona; y (c) analizar la influencia del tamaño de la ventana.

2.2 Análisis de la imagen y del problema

La imagen es de 256×256 píxeles en escala de grises y utiliza sólo 15 niveles de intensidad: del 0 al 11 (los cinco cuadrados oscuros y lo que hay dentro de ellos) y del 226 al 228 (el fondo claro). Los objetos ocultos se diferencian de su cuadrado por apenas unos pocos niveles de gris (por ejemplo 0 contra 10), diferencia imperceptible para el ojo humano.

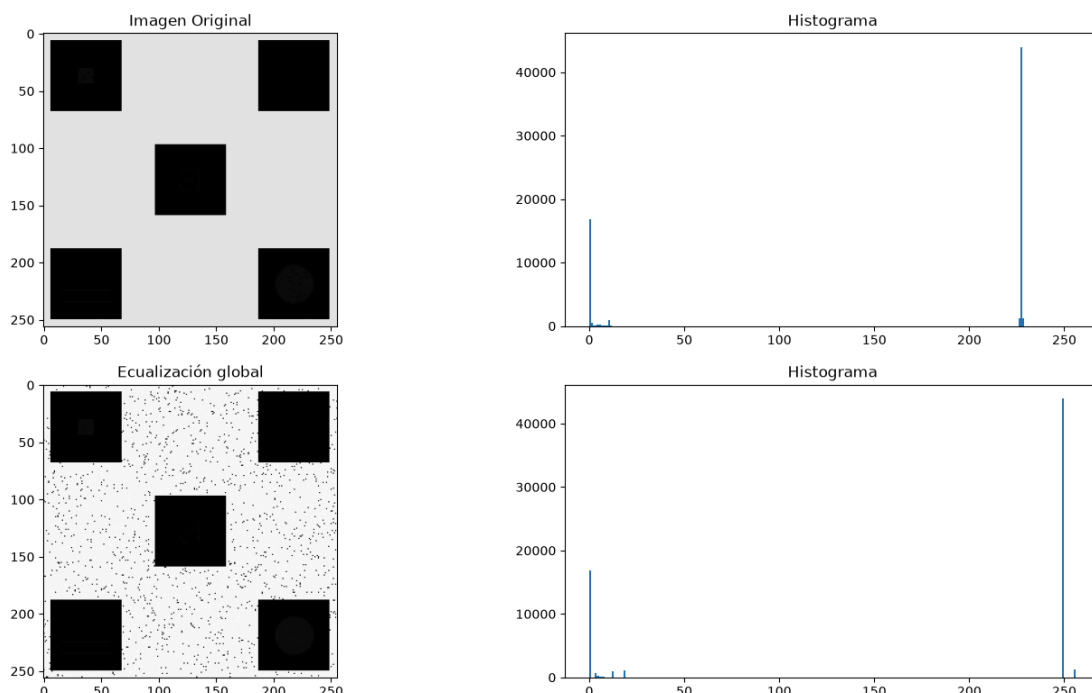


Figura 1. Imagen original y resultado de la ecualización global con su histograma. El histograma tiene dos grupos muy separados; la ecualización global separa esos dos grupos pero no realiza las diferencias dentro de cada uno, por lo que los detalles siguen ocultos.

Como el histograma global está dominado por dos grandes grupos de píxeles, la transformación global asigna casi todo el rango de salida a separar el fondo de los cuadrados, y los niveles 0–11 siguen siendo prácticamente negros. Esto justifica el uso de un método local, en el que el histograma de cada ventana sólo contiene los niveles de esa zona.

2.3 Técnica utilizada – punto a

Se implementó la función `ecualizacion_local(img, M, N)`:

- **Bordes:** para que la ventana no se salga de la imagen en los píxeles del borde se agrega un marco con `cv2.copyMakeBorder`. Se eligió `BORDER_REPLICATE` (sugerido en el enunciado), que repite los valores del borde y evita introducir intensidades artificiales (por ejemplo ceros) que alterarían el histograma local.
- **Recorrido:** gracias al borde agregado, la ventana `img_pad[i:i+M, j:j+N]` queda centrada en el píxel (i, j) de la imagen original, cuyo valor está en la posición $[m, n]$ de la ventana.
- **Transformación:** se reutiliza `cv2.equalizeHist` sobre la ventana, que calcula el histograma, la función de distribución acumulada y la tabla de mapeo; de la ventana ecualizada sólo se conserva el píxel central.
- **Tamaño de ventana:** se recomiendan valores impares para que exista un píxel central. La fórmula del borde $(m \text{ y } M - m - 1)$ también funciona con valores pares.

2.4 Resultados – punto b

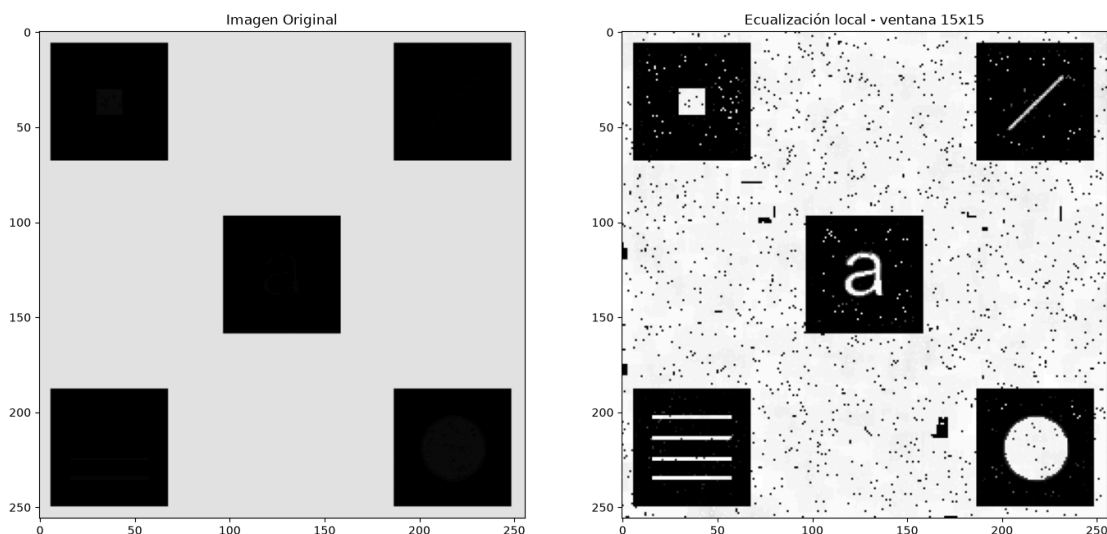


Figura 2. Imagen original y resultado de la ecualización local con ventana de 15×15 .

El resultado de la ecualización del histograma fue: en el cuadrado superior izquierdo el detalle oculto es un cuadrado en el centro, en el cuadrado superior derecho una línea diagonal, en el cuadrado central es la letra “a”, en el inferior izquierdo cuatro líneas horizontales y en el inferior derecho es un círculo.

Además, en el fondo claro aparecen puntos negros dispersos. No son un error de la implementación: el fondo, que parece uniforme, tiene variaciones de 226 a 228, y la ecualización local amplifica esas diferencias mínimas hasta hacerlas visibles (ruido amplificado).

2.5 Influencia del tamaño de la ventana – punto c

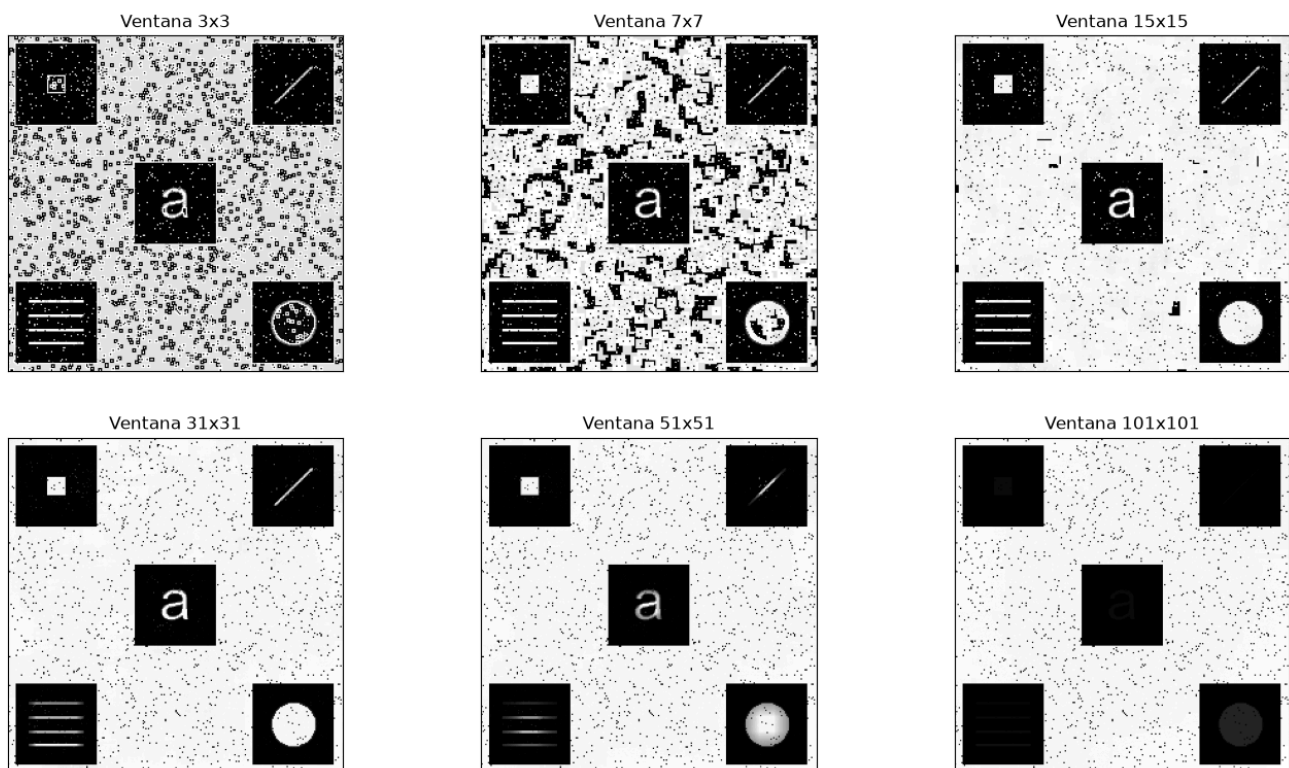


Figura 3. Resultado de la ecualización local con ventanas de 3×3, 7×7, 15×15, 31×31, 51×51 y 101×101.

En la figura 3 observamos que:

- Para las ventanas de 3x3 y 7x7 contienen muy pocos píxeles, por lo que cualquier variación mínima se lleva a los extremos. Los detalles aparecen, pero el ruido del fondo se amplifica mucho (manchas) y los bordes de los objetos quedan quebrados (el círculo aparece hueco o con manchas).
- Para las ventanas de 15x15 y 31x31 estas son más grandes que el detalle a revelar pero siguen siendo locales. Los cinco detalles se ven nítidos y completos, con un fondo poco ruidoso.

- La ventana de 51x51 empieza a incluir zonas del fondo claro; el histograma vuelve a ser bimodal y el realce disminuye. Los detalles aparecen con degradé y contraste desparejo (la «a», la diagonal y las líneas).

- Por último para la de 101x101 el comportamiento se aproxima al de la ecualización global: los detalles casi desaparecen. Con una ventana del tamaño de la imagen el resultado sería prácticamente igual al global.

2.6 Problemas encontrados

- **Píxeles del borde:** sin un marco, las ventanas de los bordes se salen de la imagen. Se resolvió con `cv2.copyMakeBorder` y `BORDER_REPLICATE`.
- **Ruido amplificado:** en zonas casi uniformes la ecualización local exagera variaciones muy pequeñas. Es una limitación propia del método y depende del tamaño de ventana.
- **Costo computacional:** se calcula una ecualización por cada píxel (65.536 ventanas). Con imágenes de 256×256 el tiempo es de pocos segundos, pero crece con el tamaño de la imagen y de la ventana.

2.7 Conclusiones

- La ecualización global no es adecuada cuando los detalles tienen intensidades similares a su fondo local, porque el histograma global pierde la información de cada zona.
- La ecualización local revela los cinco detalles ocultos: un cuadrado, una línea diagonal, la letra «a», cuatro líneas horizontales y un círculo.
- El tamaño de la ventana es el parámetro clave: debe ser mayor que el detalle a revelar y menor que la región que lo contiene. Para esta imagen, ventanas de entre 15×15 y 31×31 dan los mejores resultados.
- Ventanas chicas amplifican el ruido; ventanas grandes tienden a la ecualización global.

3. Problema 2 – Validación de planilla de calificaciones

3.1 Descripción del ejercicio

Se dispone de cuatro planillas de calificaciones escaneadas (grade_sheet_1..4.png) y una plantilla vacía de referencia. Cada planilla tiene 20 registros con los campos Legajo, Nombre y Apellido, Parcial 1, 2 y 3 y Condición Final. Se debe validar automáticamente cada campo según las siguientes restricciones:

- **Legajo:** Exactamente 8 caracteres formando una única palabra.
- **Nombre y apellido:** Al menos dos palabras y no más de 12 caracteres.
- **Parcial 1, 2 y 3:** 1 o 2 caracteres consecutivos (sin espacios).
- **Condición Final:** Un único carácter.

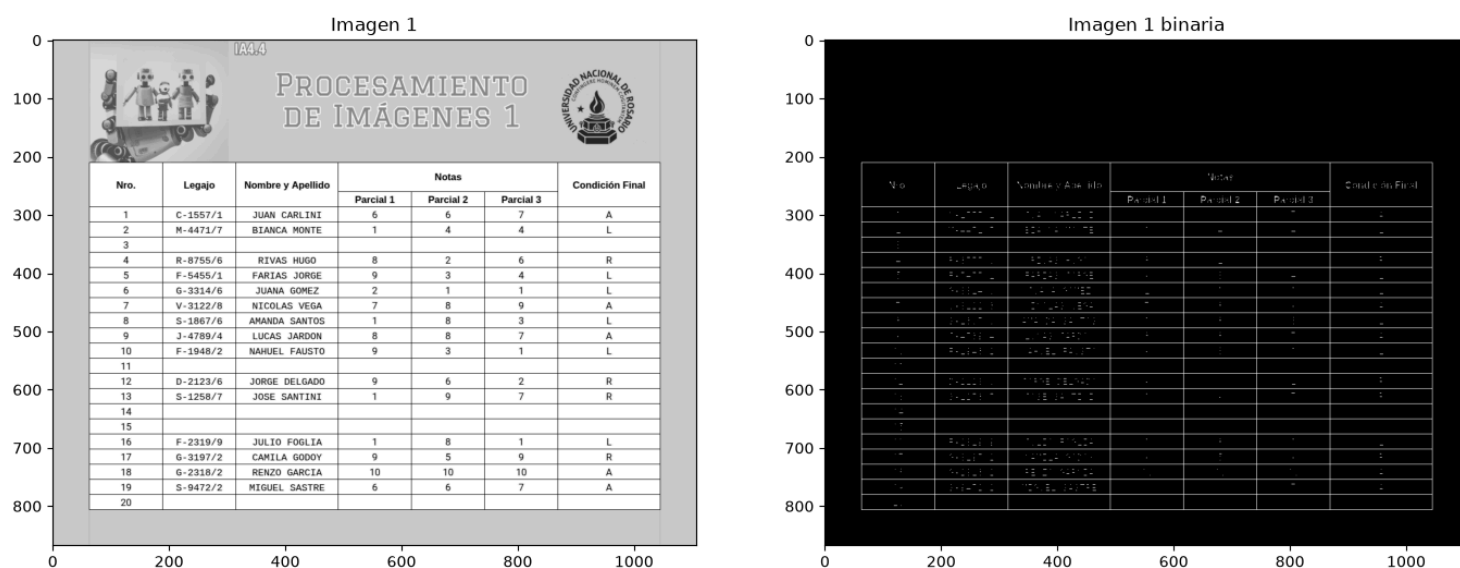


Figura 4. Planilla 1 en escala de grises y su umbralización con `img == 0`: las líneas de la tabla y el texto quedan en blanco.

3.2 Análisis inicial

- **Tamaños distintos:** las cinco imágenes tienen dimensiones diferentes (la vacía 697×884; las planillas 867×1107, 869×1029, 869×1106 y 1082×917) y distinto tamaño de letra. Por eso no se pueden usar coordenadas fijas: las posiciones de la tabla deben detectarse en cada imagen.
- **Niveles de gris:** las imágenes usan todos los niveles de 0 a 255 (por el encabezado y el suavizado del texto), pero las líneas de la tabla son exactamente negras (valor 0). Por eso se umbralizó con `img == 0`.
- **Primeras pruebas:** se comenzó trabajando sobre la planilla vacía y la planilla 1 con umbrales fijos elegidos a mano (≥ 400 y luego ≥ 548 píxeles por columna). Funcionaban en algunas planillas pero no eran generales: en la planilla vacía la

línea más larga suma 482 y en la planilla 4 suma 848. Esto llevó a usar un umbral relativo a cada imagen.

3.3 Detección de la tabla – punto a

3.3.1 Columnas

Se suman los píxeles negros de cada columna (`np.sum(img == 0, axis=0)`). Las líneas verticales de la tabla atraviesan toda su altura, por lo que su suma es mucho mayor que la de las columnas con texto (≤ 56). Como umbral se usa el segundo valor más alto de la suma (`np.unique(img_cols)[-2]`), que se adapta a cada planilla: en todas detecta exactamente las 8 líneas verticales que delimitan las 7 columnas.

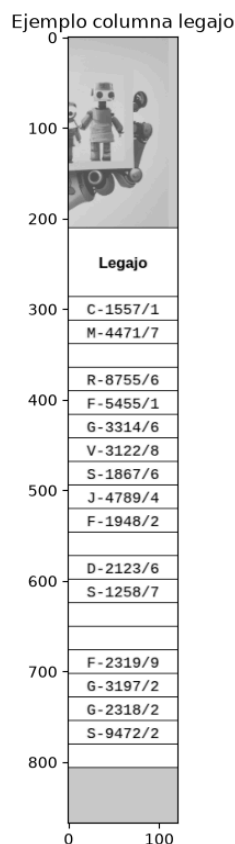


Figura 5. Columna Legajo recortada

3.3.2 Filas

Cada columna se recorta dejando 3 píxeles de margen a cada lado (para eliminar la línea vertical) y se repite el procedimiento sumando por filas. El umbral es el valor máximo de la suma, que corresponde a las líneas horizontales que cruzan todo el ancho de la columna. Se detectan 22 líneas: la superior del encabezado más las 21 que delimitan los 20 registros. Por eso se conservan sólo las últimas 21 (`indices_filas[-21:]`). Cada celda se recorta también con 3 píxeles de margen arriba y abajo.

Las celdas se guardan en `diccionario_imagenes[imagen][columna][celda]`, lo que permite acceder a cualquier celda de cualquier planilla para las etapas siguientes.

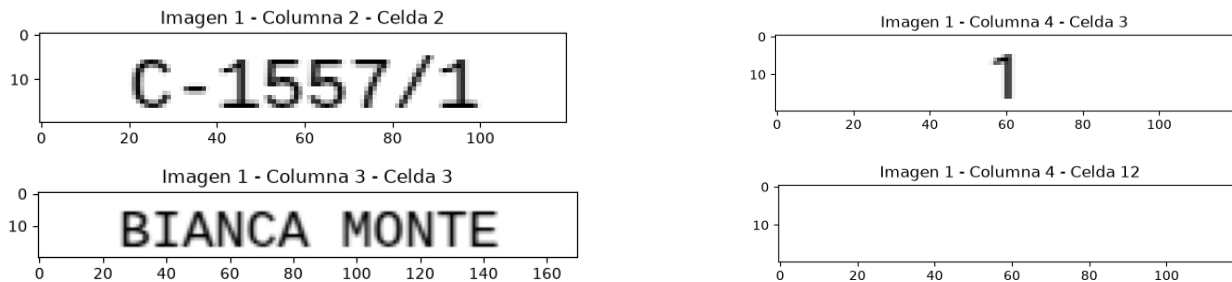


Figura 6. Mostramos varias celdas recortadas de la imagen 1 sin líneas de la tabla.

3.4 Análisis de cada celda – punto a

3.4.1 Conteo de caracteres con componentes conectadas

Cada celda se binariza e invierte (celda < 128, texto en blanco sobre fondo negro) y se aplica `cv2.connectedComponentsWithStats` con conectividad 8. Cada componente conectada corresponde a un carácter; de stats se obtienen su posición (LEFT), su ancho (WIDTH) y su área. La componente 0 (el fondo) se descarta.

3.4.2 Conteo de palabras

Las componentes se ordenan de izquierda a derecha y se calcula la distancia en blanco entre el final de un carácter y el comienzo del siguiente. Consideramos que si la distancia supera los 8 píxeles, entonces empieza una nueva palabra.

3.4.3 Filtro por área: el guión y el punto:

El enunciado sugiere eliminar componentes de área muy pequeña para descartar restos de las líneas de la tabla. Al ajustar este filtro aparecieron dos problemas:

- El guión de los legajos tiene un área de sólo 4–5 píxeles; un umbral alto lo elimina y el legajo quedaba con 7 caracteres.
- En la planilla 3 aparece la nota 1.0, cuyo punto tiene un área de 2 píxeles. Con el umbral inicial (área > 3) el punto se descarta, se leía 10 y el campo se validaba como OK cuando en realidad tiene 3 caracteres.
- Como el recorte con margen de 3 píxeles ya elimina las líneas de la tabla (no se observaron componentes de 1 píxel), cambiamos el umbral en área > 1, que conserva tanto el guión como el punto.

3.4.4 Reglas de validación y salida por consola

Con la cantidad de caracteres y de palabras de cada celda se aplican las restricciones de la sección 3.1. Una celda vacía (0 caracteres) siempre es MAL. Extracto de la salida por consola para la planilla 1:

```

--- Procesando Imagen: 1 ---

> Registro 1:
> Legajo: OK
> Nombre y Apellido: OK
> Parcial 1: OK
> Parcial 2: OK
> Parcial 3: OK
> Condición Final: OK

> Registro 2:
> Legajo: OK
> Nombre y Apellido: OK
> Parcial 1: OK
> Parcial 2: OK
> Parcial 3: OK
> Condición Final: OK

> Registro 3:
> Legajo: MAL
> Nombre y Apellido: MAL
> Parcial 1: MAL
> Parcial 2: MAL
> Parcial 3: MAL
> Condición Final: MAL

```

Figura 7. Resultado del punto a.

3.5 Alumnos no aprobados – punto b

Para validar los registros se reutilizan las mismas reglas en la función `validar_campo`, que en lugar de imprimir devuelve el resultado, y se guardan en `resultados[imagen]`. Sólo se consideran los registros con todos sus campos OK.

3.5.1 Identificación de la letra de la Condición Final

Para saber si la condición es “L” o “R”, sino características geométricas de la letra, calculadas en la función `clasificar_condicion`:

- Cantidad de agujeros: se toma la componente más grande (la letra), se invierte y se agrega un borde de fondo con `cv2.copyMakeBorder`; con `cv2.connectedComponents` (conectividad 4) se cuentan las regiones de fondo. Todas menos el fondo exterior son agujeros.
- Palo vertical izquierdo: proporción de filas de la letra que tienen píxeles en sus dos primeras columnas.
- Regla: $L = 0 \text{ agujeros} + \text{palo vertical} + \text{nada en la esquina superior derecha}$; $R = 1 \text{ agujero} + \text{palo vertical}$; $A = 1 \text{ agujero sin palo vertical (lados inclinados)}$.

3.5.2 Imagen de salida

Para cada alumno registrado correctamente y no aprobado se recorta el contenido del campo Nombre y Apellido (recortar_contenido, la unión de los rectángulos de sus componentes conectadas), se escala a una altura fija porque las planillas tienen distintos tamaños de letra, y se agrega un indicador de color: rojo = LIBRE y naranja = RECUPERA. Se genera una única imagen por planilla con cv2.putText y cv2.rectangle, guardada con cv2.imwrite.



Figura 8. Imágenes de salida del punto b para las cuatro planillas

3.6 Archivo CSV – punto c

La función `guardar_csv` escribe un archivo por planilla (resultados / validacion_imagen_X.csv) con `csv.DictWriter`. La primera columna es el ID, que corresponde al orden del registro en la planilla, y le siguen las columnas Legajo, Nombre y Apellido, Parcial 1, Parcial 2, Parcial 3 y Condición Final con los valores OK o MAL.

3.7 Aplicación a las cuatro planillas – punto d

Todo el algoritmo se ejecuta de forma cíclica: el primer bucle recorre las cuatro imágenes detectando celdas, el segundo valida e imprime los registros, y el último genera para cada planilla la imagen del punto b y el CSV del punto c.

3.8 Problemas encontrados

- **Imágenes de distinto tamaño:** los umbrales y coordenadas fijas (por ejemplo, recortar la columna Legajo en [:, 149:250]) sólo servían para una planilla. Se reemplazaron por umbrales relativos (segundo valor más alto, valor máximo) calculados en cada imagen.
- **Líneas del encabezado:** la detección de filas encuentra también la línea superior del encabezado; se resolvió quedándose con las últimas 21 líneas.
- **Restos de las líneas de la tabla dentro de las celdas:** se eliminaron recortando 3 píxeles de margen en cada borde de la celda.
- **Caracteres pequeños:** el guion y el punto tienen áreas de 2 a 5 píxeles; el umbral de área se ajustó a > 1 para no perderlos.
- **Separación de palabras:** fue necesario medir las distancias reales entre caracteres para elegir un umbral de 8 píxeles.
- **Identificar la letra de la condición:** el conteo de componentes no distingue A, L y R; se resolvió con características de forma (agujeros y palo vertical) en lugar de comparar contra plantillas.
- **Ejecución línea a línea:** con rutas relativas, el script debe ejecutarse desde la carpeta Ejercicio 2; si no, cv2.imread devuelve None sin avisar.

3.9 Limitaciones

- Se asume que las líneas horizontales de la tabla tienen 1 píxel de alto y que hay exactamente 20 registros; con líneas más gruesas habría que agrupar índices consecutivos (inicio y fin de cada línea).
- El umbral de espacio (8 px) depende del tamaño de letra; en planillas con letra mucho más grande habría que escalarlo.
- El clasificador de letras está pensado para las condiciones presentes en el trabajo (A, L y R); otra letra podría clasificarse mal.
- La validación es sintáctica (cantidad de caracteres y palabras); como pide el enunciado, no se verifica que el contenido sea válido.

3.10 Conclusiones

- La estructura de la planilla puede detectarse de forma robusta con proyecciones (sumas por filas y columnas) sobre la imagen umbralizada, siempre que los umbrales se calculen relativos a cada imagen.
- Las componentes conectadas permiten contar caracteres sin necesidad de reconocerlos, y la distancia entre ellas permite contar palabras.
- Los parámetros (umbral de área y umbral de espacio) se eligieron midiendo los datos y no a ojo.
- Características geométricas simples (agujeros y trazos verticales) alcanzan para distinguir las letras de la condición final.
- El algoritmo valida correctamente los 80 registros de las cuatro planillas y genera las imágenes y los CSV pedidos.